

list.c

```
/*
 * Simple double linked list implementation
 * created only for educational purposes
 *
 * Task: Operations should be extended with pthread_rwlocklocks
 */
#include "list.h"
#include <stdlib.h>

/*
 * function: dllist_create
 * parameter: dllist ** list: Pointer to the Pointer that should hold the address of the list
 * returns: integer, 0 success, -1 malloc failure
 * description: allocates and initializes the double linked list structure
 */
int dllist_create(dllist** list)
{
    *list = calloc(1, sizeof(dllist));
    if (*list == NULL) {
        return -1;
    }

    // Initialize RW lock
    if (pthread_rwlock_init(&((*list)->lock), NULL) != 0) {
        free(*list);
        return -1;
    }
    return 0;
}

/*
 * function: dllist_free
 * parameter: dllist * list: pointer to the list
 * returns: void
 * description: cleans up all structures
 */
void dllist_free( dllist* list)
{
    // There is no spoon
    if (list==NULL) {
        return;
    }
    while(!dllist_remove(list,0)) {}
}
```

```

    // Destroy RW lock
    pthread_rwlock_destroy(&list->lock);
    free(list);
}

/*
 * function: dllist_insert
 * parameter: dllist * list: pointer to the list, int value: value to be added, int position
 * returns: int: 0 on success, -1 on position out of range, -2 malloc failure, -3 on other error
 * description: inserts an value at position to the list
 */
int dllist_insert(dllist* list, int value, int position)
{
    if ( list == NULL || position < 0 ) {
        return -3;
    }

    // lock for writing
    pthread_rwlock_wrlock(&list->lock);

    dllist_element* list_iterator= list->first; // list element pointer
    //iterate to the right position
    while(position > 1 && list_iterator !=NULL) {
        list_iterator=list_iterator->next;
        position--;
    }

    if (position>1 || (position==1 && list_iterator==NULL)){
        // unlock and return error
        pthread_rwlock_unlock(&list->lock);
        return -1;
    }

    //allocate new element
    dllist_element* new_element = calloc(1,sizeof(dllist_element));
    if(new_element==NULL) {
        pthread_rwlock_unlock(&list->lock);
        return -2;
    }
    new_element->value=value;

    //are we first element
    if(position==0) {
        new_element->next=list->first;
        list->first=new_element;
    } else {

```

Fehlerbehandlung fehlt -0.5
auch bei jedem weiteren lock aufruf

```

        new_element->next=list_iterator->next;
        new_element->prev=list_iterator;
        list_iterator->next=new_element;
    }

    // do we have a next element
    if(new_element->next!=NULL) {
        new_element->next->prev=new_element;
    } else {
        // are we last element
        list->last=new_element;
    }

    // unlock and return success
    pthread_rwlock_unlock(&list->lock);
    return 0;
}

/*
 * function: dllist_get
 * parameter: dllist * list: pointer to the list, int position: list position the value is
 * returns: int: 0 on success, -1 on position out of range, -2 on other errors
 * description: gets an element of the list at the position
 */
int dllist_get(dllist* list, int position, int* result)
{
    if ( list == NULL || position < 0 ) {
        return -2;
    }

    // lock for reading
    pthread_rwlock_rdlock(&list->lock);

    dllist_element* list_iterator= list->first; // list element pointer

    while(position > 0 && list_iterator !=NULL) {
        list_iterator=list_iterator->next;
        position--;
    }

    if (position!=0 || list_iterator == NULL){
        // unlock and return error
        pthread_rwlock_unlock(&list->lock);
        return -1;
    }
}

```

```

        *result=list_iterator->value;

        // unlock and return success
        pthread_rwlock_unlock(&list->lock);
        return 0;
    }

    /*
     * function: dllist_remove
     * parameter: dllist * list: pointer to the list, int position: list position the value is at
     * returns: int: 0 on success, -1 on position out of range, -2 on other errors
     * description: removes an list element at the position
     */
    int dllist_remove(dllist* list, int position)
    {
        if ( list == NULL || position < 0 ) {
            return -3;
        }

        // lock for writing
        pthread_rwlock_wrlock(&list->lock);

        dllist_element* list_iterator= list->first; // list element pointer
        //iterate to the right position
        while(position > 0 && list_iterator !=NULL) {
            list_iterator=list_iterator->next;
            position--;
        }

        if (position!=0 || list_iterator == NULL){
            // unlock and return error
            pthread_rwlock_unlock(&list->lock);
            return -1;
        }

        //is there an element before us
        if (list_iterator->prev != NULL) {
            list_iterator->prev->next= list_iterator->next;
        } else {
            list->first=list_iterator->next;
        }
        if(list_iterator->next != NULL) {
            list_iterator->next->prev= list_iterator->prev;
        } else {
            list->last=list_iterator->prev;
        }
    }

```

```

    }

    free(list_iterator);

    // unlock and return success
    pthread_rwlock_unlock(&list->lock);

    return 0;
}

```

3.5/4

test.c

```

/*
 * Simple test application for the linked list application.
 * created only for educational purposes
 *
 * Task: This test application need to modified that the linked list is tested in multiple p
 */

#include "list.h"
#include <stdio.h>

void print_list(dllist *list){

    int v,r;
    int i=0;
    while(!(r=dllist_get(list,i,&v))) {
        printf("%d:",v);
        i++;
    }
    printf("\n");
}

int main()
{

    int toadd[10]= {7,0,3,0,2,1,9,2,8,4};
    int torm[4]= {0,4,1,0};
    int r;

    dllist* list;
    if(dllist_create(&list)) {
        fprintf(stderr,"Couldn't create list!");
    }
}

```

```

for(int i = 0; i < 10; i+=2) {
    if(dllist_insert(list,toadd[i],toadd[i+1])) {
        printf("couldn't add %d at %d\n",toadd[i],toadd[i+1]);
    }
}

print_list(list);

for(int k = 0; k < 4; k+=1) {
    printf("trying to remove element at position: %d \n", torm[k]);
    if((r=dllist_remove(list,torm[k]))) {
        printf("couldn't remove element at %d ret: %d \n",torm[k],r);
    }
    print_list(list);
}

for(int i = 0; i < 3; i+=2) {
    if(dllist_insert(list,toadd[i],toadd[i+1])) {
        printf("couldn't add %d at %d\n",toadd[i],toadd[i+1]);
    }
}

print_list(list);

dllist_free(list);
}

```

test_all.c nicht vorhanden 0/2

Übungsblatt 5

Montag, 5. Januar 2026 20:38

Yunis Drescher, 827130 Joshua Neunert, 822456 Konrad Gose, 828440

Aufgabe 5.2: Synchronisation

- a) Worin besteht der Unterschied zwischen dem Java- Monitor- und dem Hoare-Monitorkonzept?
- b) Die folgende Umsetzung der **put**- und **get**-Funktionen des Java-Cubbyhole-Beispiels ist fehlerhaft. Wie kann man den Fehler beheben?

```
public synchronized int get() {
    if (available == false) {
        try {
            wait();
        } catch (InterruptedException e) {
        }
    }
    available = false;
    notify();
    return contents;
}

public synchronized void put(int value) {
    if (available == true) {
        try {
            wait();
        } catch (InterruptedException e) {
        }
    }
    contents = value;
    available = true;
    notify();
}
```

- c) Überdies kann es auch in der korrigierten Lösung zu Livelocks kommen. Geben Sie ein Beispielszenario dafür an! (1 + 1 + 1 Punkte)

a)

Beim Java-Monitor wird durch `notify()` ein Thread wieder bereit gemacht, aber nicht direkt ausgeführt. Der Thread der `notify()` ausgeführt hat nehält den Monitor solange bis er ihn verlässt. Der geweckte Thread arbeitet also irgendwann später.

Beim Hoare-Monitor wird der Thread der durch `signal(x)` aufgerufen wird sofort aktiv und der Thread der `signal(x)` aufruft wird blockiert. Wenn der geweckte Thread den Monitor verlässt, kann der Signal sendende Thread weiter arbeiten. Dadurch entsteht der Unterschied, dass beim Hoare Monitor der geweckte Thread garantiert seine Bedingung erfüllt hat, während beim Java-Monitor die Bedingung sich wieder geändert haben kann. Daher muss beim Java-Monitor immer wieder mit `while` die Bedingung geprüft werden.

Hoare verschiedene Bedingungsvariablen mit eigener WS,
Java nur eine WS

0.5/1

c)

Da es sich um Java handelt, müssen wir die Verfügbarkeit nicht nur einmal mit `"if"` prüfen sondern mit einer `while`-schleife. Also:

Zeile 2: while (available == false) {
Zeile 12: while (available == true) {
Der Rest des codes kann gleich bleiben.

1/1

d)

Gehen wir davon aus das es mehr als zwei Threads gibt. Da nur notify() und nicht notifyAll() genutzt wird, kann es sein dass ein Thread geweckt wird der direkt wieder blockiert wird. Dieser Thread kann dann kein neues notify() ausführen und alles steht still. Beispiel:

P1, P2, P3

P1, P2 wollen put() nutzen, P2 will get() nutzen.

Wenn available = false, dann wartet P3 in get(). P1 ruft nun put() auf und schreibt seinen wert. Danach setzt er available = true und notify() weckt P2. P2 sieht available = true und geht in wait. Nun ist P1 abgeschlossen und P2 und P3 stecken in der wait() schleife.

livelock

1/1

Aufgabe 5.3: Buddy-System

Ein Betriebssystem verwende das Buddy-System für die Speicherverwaltung mit 2^k KB ($k \geq 0$) großen Blöcken. Der Rechner ist mit 2048 KByte RAM ausgestattet, die vollständig im Buddy-System verwaltet werden.

Die Prozesse A, B, C und D fordern Speicher in folgender Reihenfolge an:

- i) A fordert 236 KByte an.
 - ii) B fordert 144 KByte an.
 - iii) C fordert 76 KByte an.
 - iv) D fordert 456 KByte an.
- a) Zeichnen Sie, nach den Schritten ii) , iii) und iv) die Speicherbelegung ausgehend vom komplett freien und nicht in Blöcke eingeteilten Speicher.
 - b) Wieviele Blöcke sind noch übrig? Wie groß ist dann der größte verfügbare Speicherbereich?
 - c) Geben Sie den aktuellen Verschnitt an!

Nacheinander geben dann die Prozesse B, A, D und C in dieser Reihenfolge ihren Speicher frei.

- d) Zeichnen Sie, ausgehend vom Ergebnis von a), für jeden Schritt die Speicherbelegung. Geben Sie nach jeder Speicherfreigabe den größten zur Verfügung stehenden Block an!

a)

Aufgabe 5.3: 4/4

a) 2/2

Aktion:

o)	2048 KByte						
i)	256 Kbyte	256 KByte	512 Kbyte		1024 KByte		
	A:236	✓					
ii)	256 Kbyte	256 KByte	512 Kbyte		1024 KByte		
	A:236	B:144	✓				
iii)	256 Kbyte	256 KByte	128 KB	128 KB	256 Kbyte	1024 KByte	
	A:236	B:144	C:76	✓			
iv)	256 Kbyte	256 KByte	128 KB	128 KB	256 Kbyte	512 Kbyte	512 Kbyte
	A:236	B:144	C:76			D: 456	✓

0,5/0,5

b)

Es sind noch 3 Blöcke übrig, wovon der größte Block 512 KByte groß ist ✓

0,5/0,5

c)

Der Verschnitt ist der ungenutzte Speicher in jedem Benutzen Speicherblock, also:

$$(256 - 236) + (256 - 144) + (128 - 76) + (512 - 456) = 240 \quad \checkmark$$

1/1 d)

Entfernen:

		256 KByte	256 KByte	128 KB 128 KB	256 KByte	512 KByte	512 KByte
B		A:236	✓	C:76		D: 456	größter freier Speicher
		512 Kbyte	✓	128 KB 128 KB	256 KByte	512 KByte	512 KByte
A		größter freier Speicher		C:76		D: 456	größter freier Speicher
		512 Kbyte		128 KB 128 KB	256 KByte	1024 KByte	
D				C:76		größter freier Speicher	✓

Aufgabe 5.4: Gutartigkeit

Zeigen Sie, daß LRU ein gutartiger Seitenersetzungsalgorithmus ist!

(2 Punkte)

Beweis über Stack-Algorithmus:

Ein Algorithmus ist gutartig, wenn er ein Stack-Algorithmus ist.
 Ein Algorithmus ist ein Stack-Algorithmus, wenn es eine feste Prioritätsordnung gibt mit der die Seiten eingeladen werden.

Also gilt in Bezug auf Gutartigkeit, dass:

- Der Speicher der Größe m enthält immer die m höchstpriorisierten Seiten
- Bei größerem Speicher werden nur "unten" zusätzliche Seiten ergänzt

Da LRU eine feste Prioritätsordnung hat, höchste Priorität der Ersetzung für die am längsten nicht genutzte Seite und somit die Anforderung eines Stack-Algorithmus erfüllt ist er gutartig

Aufgabe 5.5: Speicherverwaltung Seitenfehler 2/2

Die Behandlung eines Seitenfehlers erfordert i. A. Maßnahmen zur Ersetzung einer Seite in einem Seitenrahmen (Kachel) des Arbeitsspeichers. Dazu muss zunächst eine andere Seite aus dem Arbeitsspeicher verdrängt werden.

Bei der Ausführung eines Benutzerprogramms tritt folgende Sequenz von Seitenzugriffen auf:

7 6 2 7 5 5 3 2 7 4 5

Bestimmen Sie die minimale Anzahl der Seitenfehler, die auftreten kann! Gehen Sie dabei von 4 Seitenrahmen aus und begründen Sie Ihre Antwort. (2 Punkte)

Die Minimale Anzahl von Seitenfehler erhält man über den Belady-Algorithmus, da er die Seiten die man als nächstes Braucht kennt und somit immer optimal löscht. Dieser ist aber nur möglich wenn man die Sequenz kennt bevor sie passiert.

SW	7	6	2	7	5	5	3	2	7	4	5
1	7	7	7	7	7	7	7	7	7	4	4
2		6	6	6	6	6	3	3	3	3	3
3			2	2	2	2	2	2	2	2	2
4					5	5	5	5	5	5	5
SF	*	*	*		*		*			*	

Die minimale Anzahl der Seitenfehler ist also 6

3/3 Aufgabe 5.6: Speicherverwaltung Algorithmen

Es steht ein physikalischer Adressraum mit 4 Seitenrahmen zur Verfügung. Arbeiten Sie die angegebene Seitenzugriffsfolge gemäß der Seitenersetzungsstrategien:

- FIFO (First-In-First-Out),
- LRU (Least-Recently-Used),
- Auslagern einer Seite, die nicht zum Working-Set gehört (Working-Set-Parameter $T=3$).

ab, und vergleichen Sie diese anhand der Seitenfehleranzahl!

2 7 6 7 1 4 7 2 5 5 1

In denjenigen Fällen, in denen Nicht-Determiniertheit vorliegt, ist stets die Seite mit der kleinsten Seitenrahmennummer zu wählen, die das jeweilige Verfahren erfüllt.

(3 Punkte)

FIFO											
SW	2	7	6	7	1	4	7	2	5	5	1
Jung	2	7	6	6	1	4	4	2	5	5	5
		2	7	7	6	1	1	4	2	2	2
			2	2	7	6	6	1	4	4	4
Alt					2	7	7	6	1	1	1
SF	*	*	*		*	*		*	*		



LRU											
SW	2	7	6	7	1	4	7	2	5	5	1
1	2	2	2	2	2	4	4	4	4	4	1
2		7	7	7	7	7	7	7	7	7	7
3			6	6	6	6	6	2	2	2	2
4					1	1	1	1	5	5	5
SF	*	*	*		*	*		*	*		*



T=3											
SW	2	7	6	7	1	4	7	2	5	5	1
1	2	2	2	2	2	4	4	4	4	4	1
2		7	7	7	7	7	7	7	7	7	7
3			6	6	6	6	6	2	2	2	2
4					1	1	1	1	5	5	5
SF	*	*	*		*	*		*	*		*



FIFO hat 7 Seitenfehler und LRU und Working Set haben 8 Seitenfehler.

